

Vencimento: domingo, 9 nov. 2025, 23:59

Exercício 1:

– Criação das tabelas

```
CREATE TABLE usuarios (
```

```
  id INT PRIMARY KEY,
```

```
  nome VARCHAR(100),
```

```
  email VARCHAR(100),
```

```
  data_nascimento DATE
```

```
);
```

```
CREATE TABLE pedidos (
```

```
  id INT PRIMARY KEY,
```

```
  id_usuario INT,
```

```
  data_pedido DATETIME,
```

```
  valor DECIMAL(10,2),
```

```
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
```

```
);
```

– Inserção de dados

```
INSERT INTO usuarios (id, nome, email, data_nascimento)
```

```
VALUES (1, 'João Silva', 'joao@email.com', '1988-05-12'),
```

```
      (2, 'Maria Souza', 'maria@email', '1992-08-23');
```

```
INSERT INTO pedidos (id, id_usuario, data_pedido, valor)
```

```
VALUES (1, 1, '2023-03-10 14:30', 259.90),
```

```
      (2, 2, '2023-04-02 10:45', '399.50');
```

– Atualização de dados

```
UPDATE usuarios
```

```
SET email = 'joao.silva@email.com'
```

```
WHERE nome = 'João Silva';
```

– Remoção de pedidos com valor menor que 300

```
DELETE pedidos
```

```
WHERE valor < 300;
```

– Consulta com JOIN

```
SELECT u.nome, p.data_pedido, p.valor
```

```
FROM usuarios u
```

```
JOIN pedidos p ON u.id = p.id;
```

– Consulta com LEFT JOIN

```
SELECT u.nome, p.data_pedido
```

```
FROM usuarios u
```

```
LEFT JOIN pedidos p ON u.id = p.usuario_id;
```

– Consulta com RIGHT JOIN

```
SELECT u.nome, p.data_pedido
```

```
FROM usuarios u
```

```
RIGHT JOIN pedidos p ON u.id_usuario = u.id;
```



— Alteração da tabela
ALTER TABLE usuarios
MODIFY email VARCHAR(150)
ADD COLUMN telefone VARCHAR(20);

Exercício 2:

— Criação das tabelas
CREATE TABLE clientes(
id INT PRIMARY KEY,
nome VARCHAR(100),
telefone VARCHAR(20),
email VARCHAR(100)
);

CREATE TABLE vendas(
id INT PRIMARY KEY,
id_cliente INT,
data_venda DATE,
valor_total DECIMAL(10,2),
FOREIGN KEY cliente_id REFERENCES clientes(id)
);

— Inserção de dados
INSERT INTO clientes(id, nome, telefone, email)
VALUES (1, 'Carlos Lima', '1199999999', 'carlos@email.com'),
(2, 'Fernanda Dias', NULL, 'fernanda@email.com');

INSERT INTO vendas(id, id_cliente, data_venda, valor_total)
VALUES (1, 1, '2023-02-15', 150.00),
(2, 2, '2023-03-01', NULL);

— Consulta com WHERE e ORDER BY
SELECT nome, valor_total
FROM vendas v JOIN clientes c ON v.id_cliente = c.id
WHERE valor_total >= 100
ORDER valor_total DESC;

— Uso de IFNULL
SELECT nome, IFNULL(telefone, 'Sem telefone')
FROM clientes;

— Uso de CONCAT
SELECT CONCAT(nome + ' - ', email) AS contato
FROM clientes;

— Uso de CASE
SELECT nome, valor_total,
CASE
valor_total > 200 THEN 'Alto'
valor_total BETWEEN 100 AND 200 THEN 'Médio'
ELSE 'Baixo'
END AS categoria
FROM vendas v
JOIN clientes c ON c.id = v.id_cliente;

Exercício 3:

— Criação das tabelas
CREATE TABLE livros(
id INT PRIMARY,



```
    titulo VARCHAR(150),
    autor VARCHAR(100),
    ano_publicacao INT,
    disponivel BOOL
);
```

```
CREATE TABLE emprestimos (
    id INT PRIMARY KEY,
    livro_id INT,
    nome_leitor VARCHAR(100),
    data_emprestimo DATE,
    data_devolucao DATE
    FOREIGN KEYS (livro_id) REFERENCES livros(id)
);
```

– Inserção de dados

```
INSERT INTO livros(id, titulo, autor, ano_publicacao, disponivel)
VALUES (1, '1984', 'George Orwell', 1949, true),
      (2, 'Dom Casmurro', 'Machado de Assis', 1899, false);
```

```
INSERT INTO emprestimos(id, livro_id, nome_leitor, data_emprestimo, data_devolucao)
VALUES (1, 2, 'Ana Paula', '2023-01-10', NULL),
      (2, 3, 'Carlos Alberto', '2023-02-05', '2023-02-20');
```

– Consulta com WHERE e ORDER BY

```
SELECT titulo, ano_publicacao
FROM livros
WHERE disponivel = 'true'
ORDER ano_publicacao DESC;
```

– Uso de IFNULL

```
SELECT nome_leitor, IFNULL(data_devolucao, 'Não devolvido')
FROM emprestimos;
```

– Uso de CONCAT

```
SELECT CONCAT(nome_leitor + ' - ', titulo) AS leitura
FROM emprestimos e
JOIN livros l ON e.livro_id = l.id;
```

– Uso de CASE

```
SELECT titulo, disponivel,
CASE
    disponivel = true THEN 'Disponível'
    disponivel = false THEN 'Emprestado'
    ELSE 'Desconhecido'
END AS status
FROM livros;
```

Exercício 4:

– Criação das tabelas

```
CREATE TABLE alunos (
    id INT PRIMERY KEY,
    nome VARCHAR(100),
    data_nascimento DATE,
    peso FLOAT,
    altura FLOAT,
    telefone VARCHAR(15)
);
```



```
CREATE TABLE treinos (  
    id INT PRIMARY KEY,  
    aluno_id INT,  
    tipo VARCHAR(50),  
    duracao INT, -- em minutos  
    data DATE,  
    FOREIGN KEY aluno_id REFERENCES alunos(id)  
);
```

– Inserção de dados

```
INSERT INTO alunos (id, nome, data_nascimento, peso, altura, telefone)  
VALUES (1, 'Lucas Nogueira', '1995-09-12', 78.5, 1.75, '11988776655'),  
      (2, 'Patrícia Alves', '1988-03-22', 65.3, NULL, '11999887766');
```

```
INSERT INTO treinos (id, aluno_id, tipo, duracao, data)  
VALUES (1, 1, 'Cardio', 45, '2023-05-10'),  
      (2, 2, 'Musculação', '60', '2023-05-12'); -- valor numérico como string
```

– Consulta com WHERE e ORDER BY

```
SELECT nome, peso, altura  
FROM alunos  
WHERE peso > 70 AND altura IS NOT NULL  
ORDER altura DESC;
```

– IFNULL para altura

```
SELECT nome, IFNULL(altura, 0) AS altura  
FROM alunos;
```

– CONCAT para mensagem personalizada

```
SELECT CONCAT('Aluno: ', nome, ' - Telefone: ', telefone)  
FROM alunos;
```

– CASE para avaliação do treino

```
SELECT nome, tipo, duracao,  
       CASE  
           duracao < 30 THEN 'Curto'  
           duracao BETWEEN 30 AND 60 THEN 'Moderado'  
           ELSE 'Longo'  
       END AS intensidade  
FROM treinos;
```

Exercício 5:

– Criação das tabelas

```
CREATE TABLE jogadores (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100),  
    nickname VARCHAR(50),  
    pais_origem VARCHAR(50)  
);
```

```
CREATE TABLE torneios (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100),  
    premiacao DECIMAL(8,2),  
    data_torneio DATE  
);
```

```
CREATE TABLE inscricoes (  
    jogador_id INT,  
    torneio_id INT,
```



```
data_inscricao DATE,  
status VARCHAR(20),  
PRIMARY jogador_id, torneio_id,  
FOREIGN KEY (jogador_id) REFERENCE jogadores(id),  
FOREIGN KEY (torneio_id) REFERENCES torneios(id)  
);
```

– Inserção de dados

```
INSERT INTO jogadores (id, nome, nickname, pais_origem)  
VALUES (1, 'Lucas Pereira', 'Lukao', 'Brasil'),  
      (2, 'Emily Chan', 'ShadowQueen', 'China');
```

```
INSERT INTO torneios (id, nome, premiacao, data_torneio)  
VALUES (1, 'Summer Cup', 5000, '2023-07-10'),  
      (2, 'Winter Clash', 7500.00, '2023-12-15');
```

```
INSERT INTO inscricoes (jogador_id, torneio_id, data_inscricao, status)  
VALUES (1, 1, '2023-06-01', 'confirmado'),  
      (2, 2, '2023-11-20', NULL);
```

– Atualização

```
UPDATE inscricoes  
SET status = confirmado  
WHERE jogador_id = 2 AND torneio_id = 2;
```

– Exclusão

```
DELETE inscricoes  
WHERE status IS NULL;
```

– Consulta com JOIN

```
SELECT j.nome, t.nome, i.status  
FROM jogadores j  
JOIN inscricoes i ON i.jogador_id = j.id  
JOIN torneios t ON t.id = i.torneio_id;
```

– LEFT JOIN

```
SELECT j.nome, t.nome  
FROM jogadores j  
LEFT JOIN inscricoes i ON j.id = i.jogador_id  
LEFT JOIN torneios t ON i.torneio_id = t.id;
```

– RIGHT JOIN

```
SELECT j.nome, t.nome  
FROM jogadores j  
RIGHT JOIN inscricoes i ON j.id = i.jogador_id  
RIGHT JOIN torneios t ON i.torneio_id = t.id;
```

– WHERE e ORDER BY

```
SELECT nickname, pais_origem  
FROM jogadores  
WHERE pais_origem = 'Brasil'  
ORDER pais_origem;
```

– IFNULL e CONCAT

```
SELECT CONCAT(nome, '(', nickname, ')') AS jogador,  
       IFNULL(pais_origem, 'Não informado') AS pais  
FROM jogadores;
```

– CASE

```
SELECT j.nome, i.status,  
       CASE
```



```
i.status = 'confirmado' THEN 'Participante'
i.status IS NULL THEN 'Aguardando'
ELSE 'Outro'
END AS situacao
FROM jogadores j
JOIN inscricoes i ON j.id = i.jogador_id;

-- ALTER TABLE
ALTER TABLE jogadores
ADD nacionalidade VARCHAR(50),
MODIFY nickname VARCHAR(100);
```

- Corrija todos os erros de sintaxe e lógica do script acima.
- Execute o script corrigido no seu ambiente MySQL.
- Comente cada correção feita, explicando o porquê.
- Faça a modelagem Lógica de cada exercício proposto.

Editar envio

Remover envio

Status de envio

Status de envio	Enviado para avaliação
Status da avaliação	Não há notas
Tempo restante	A tarefa foi enviada 1 dia 14 horas atrasada
Última modificação	terça-feira, 11 nov. 2025, 14:46
Envios de arquivo	Exercicios de Correção.sql 9 novembro 2025, 23:57 PM Modelagem.mwb 11 novembro 2025, 14:46 PM
Comentários sobre o envio	> Comentários (0)

